C. Odd Process

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1300 | Tags: greedy, sortings | https://codeforces.com/problemset/problem/2176/C

You have n coins with denominations $a_1, a_2, \ldots, a_n$ and a natural number k. You also have a bag, which is initially empty, where you can place coins. You need to perform exactly k actions. In each action, you take one coin from those you have left and put it in your bag. After that, you can no longer take that coin.

At the same time, you have a cat that loves even numbers, so every time the sum of the denominations of the coins in your bag becomes even, your cat empties the bag, meaning it takes all the coins to a place known only to it, and the bag is empty again. Note that the bag is emptied every time the sum becomes even during the process of adding coins, not just at the very last moment.

Let your score be the sum of the denominations of the coins in the bag. Your task is to perform k actions such that your final score is maximized. Find the answer for all $1 \le k \le n$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \le n \le 2 \cdot 10^5$) — the number of coins you have.

The second line of each test case contains n natural numbers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the denominations of the coins.

It is guaranteed that the sum of n across all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n numbers — the maximum possible score that can be achieved by performing exactly k actions for all k from 1 to n.

Example

Input

```
6
3
1 1 1
3
1 2 3
5
4 1 3 1 2
5
4 2 3 1 3
3
4 1 2
3
4 2 2
```

Output

```
1 0 1
3 5 0
3 7 9 7 9
3 7 9 7 9
1 5 7
0 0 0
```

Note

In the first set of input data, you have coins with denominations [1,1,1].

- k=1: in this case, the sum of the denominations in the bag is 1, regardless of the chosen coin. The final score is also 1.
- k=2: in this case, the sum of the denominations in the bag is initially 1, regardless of the choice of coin. When choosing the second coin, the sum will be 2, regardless of the choice of coin, and thus the bag will be emptied. The final score is 0.
- k=3: in this case, when choosing the first two coins, the sum will be 2 and the bag will be emptied, after which the sum will be 0. When choosing the third coin, the sum will become 1.

In the second set of input data, you have coins with denominations [1,2,3].

- k=1: in this case, when choosing the coin with denomination 2, the bag will be emptied and the final score will be 0. When choosing the coin with denomination 1 or 3, the final scores will be 1 and 3, respectively. The maximum possible final score is 3.
- k=2: in this case, when choosing two coins 1 and 3 in any order, their sum will be 4, after which the bag will be emptied and the final score will be 0. However, when choosing 3 as the first coin, the score will be 3. Then you can choose, for example, coin 2, and the final score will become 5.
- k=3: in this case, the sum of all coins will be 6, and since each time the bag is emptied it does not change the parity of the accumulated sum, the final score is 0.